

Information–Theoretic Barriers for Collatz Proofs

Craig Alan Feinstein

Contents

1	Collatz map and parity vectors	4
2	Affine formula characterisation	5
3	Two-adic invariance	8
4	Every parity vector is realisable	11
5	First arrival and parity prefixes	19
6	Proof system setup	20
7	The unprovability theorem	20

Information–Theoretic Barriers for Collatz Proofs

Craig A. Feinstein

Machine-Checked Formalization in Isabelle/HOL

Abstract

In an earlier paper, *The Collatz $3n + 1$ Conjecture is Unprovable* (2012), the author argued that any proof of the Collatz Conjecture would need to contain arbitrarily large amounts of information about the parity pattern of Collatz trajectories, making a finite proof impossible. The present work formalises that idea in Isabelle/HOL. Under explicit assumptions about how proofs store and preserve information, we prove that no finite proof can establish the required convergence property. This yields a machine-checked information-theoretic barrier theorem inspired by the earlier argument.

Provenance

The present development formalises the information-theoretic ideas underlying the author’s earlier paper:

C. A. Feinstein, “*The Collatz $3n + 1$ Conjecture is Unprovable*”, arXiv:math/0312309; *Global Journal of Science Frontier Research*, Mathematics and Decision Sciences, Volume 12, Issue 8 (2012), 13–15.

The assumptions required for that argument are made explicit and formalised within Isabelle/HOL, yielding a machine-checked conditional theorem showing that no finite proof can exist within the corresponding class of proof systems. These assumptions are motivated by structural properties of the Collatz map.

The author of this formalisation received assistance from two AI systems — ChatGPT (OpenAI) and Claude (Anthropic). Their assistance consisted of drafting and refining explanatory text, improving the readability of the introduction and comments, and helping diagnose or structure Isabelle/HOL proof scripts.

Main goal

The present development establishes an information-theoretic barrier for proof methods that explicitly store the required parity information.

High-level strategy

Suppose a finite certificate of length L proves the Collatz conjecture. There are 2^{L+1} possible lists of $L + 1$ parity bits. For each list, we can choose a positive starting value whose trajectory has exactly those first $L + 1$ parity bits and has not reached 1 by step L . The certificate proves convergence for every chosen starting value. By soundness, each trajectory therefore reaches 1 at some later step. The completed parity trace begins with the chosen list.

These completed traces are distinct because their initial patterns differ. Injectivity therefore gives 2^{L+1} distinct encodings, all of which must occur inside the same certificate by trace specification. Each encoding must have length at most L , but there are only $2^{L+1} - 1$ binary strings of that length or shorter. This contradiction rules out a finite universal certificate satisfying the stated assumptions.

Structure of the formalisation

- 1. Collatz map and parity vectors** We define the Collatz function T and formalise parity vectors as computational traces of the iterative process.

2. **Affine formula characterisation** We show that $T^{(k)}(n)$ admits an affine representation $(3^s n + c)/2^k$, and prove that the parameters (k, s, c) are uniquely determined by the parity vector.
3. **Two-adic invariance** We establish a key invariance property: adding 2^k to a starting value does not affect the first k parity bits. This property underlies the realisability construction.
4. **Every parity vector is realisable** We prove that every finite binary string occurs as the parity vector of some starting value. This shows that arbitrarily complex computational traces are inherent to the Collatz dynamics.
5. **Proof system setup** We introduce an abstract model of proof certificates based on bitstrings, substring containment, and information-theoretic compressibility.
6. **The original information-barrier argument** We formalise the original information-theoretic argument for the Collatz conjecture, with its proof-system requirements stated explicitly.

```
theory Collatz_Information_Barriers
  imports Main
begin
```

1 Collatz map and parity vectors

The Collatz function

We define the Collatz function T by

$$T(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ (3n + 1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

```
definition T :: "nat ⇒ nat" where
  "T n = (if even n then n div 2 else (3*n + 1) div 2)"
```

```
abbreviation Tpow :: "nat ⇒ nat ⇒ nat"
  where "Tpow k n ≡ (T ^^ k) n"
```

The Collatz conjecture

The formulation of the conjecture used in the earlier paper, and throughout this development, is:

$$\forall n > 0. \exists k. T^{(k)}(n) = 1.$$

The parity vector as computational trace

The parity vector $\text{parity_vec } n \ k$ records the sequence of even and odd values encountered during the first k iterations starting from n :

$$\text{parity_vec } n \ k = [\text{odd}(n), \text{odd}(T(n)), \text{odd}(T^{(2)}(n)), \dots, \text{odd}(T^{(k-1)}(n))].$$

This parity vector represents the *computational trace*: the sequence of branching decisions made while repeatedly applying the Collatz map.

Example

For $n = 27$ we have:

$$T(27) = 41, \quad T^{(2)}(27) = 62, \quad T^{(3)}(27) = 31.$$

Thus:

$$\begin{aligned} \text{parity_vec } 27 \ 4 &= [\text{odd}(T^{(0)}(27)), \text{odd}(T^{(1)}(27)), \text{odd}(T^{(2)}(27)), \text{odd}(T^{(3)}(27))] \\ &= [\text{odd}(27), \text{odd}(41), \text{odd}(62), \text{odd}(31)] \\ &= [\text{True}, \text{True}, \text{False}, \text{True}]. \end{aligned}$$

definition $\text{collatz_conjecture} :: \text{bool where}$
 $\text{"collatz_conjecture} \longleftrightarrow (\forall n > 0. \exists k. \text{Tpow } k \ n = 1) \text{"}$

definition $\text{parity_vec} :: \text{"nat} \Rightarrow \text{nat} \Rightarrow \text{bool list}"$
 $\text{where "parity_vec } n \ k = \text{map } (\lambda i. \text{odd } (\text{Tpow } i \ n)) \ [0..<k]"$

lemma $\text{length_parity_vec[simp]: "length (parity_vec } n \ k) = k"$
 $\text{by (simp add: parity_vec_def)}$

2 Affine formula characterisation

After k iterations of the Collatz map T , the result admits an affine representation:

$$T^{(k)}(n) = \frac{3^s n + c}{2^k}.$$

Here:

- k is the number of iterations.
- s counts the number of odd values in the parity vector, that is, the number of steps of the form $(3n + 1)/2$.
- c is a constant determined by the specific parity sequence.

Intuition

Each odd step multiplies the current value by 3, adds 1, and then divides by 2, while each even step simply divides by 2. After k steps, the cumulative effect is multiplication by $3^s/2^k$ and the addition of $c/2^k$.

Uniqueness

The parameters (k, s, c) are uniquely determined by the parity vector. In particular, the parity vector encodes all computational information needed to reconstruct the affine formula for $T^{(k)}(n)$.

```
fun params0 :: "bool list ⇒ nat × nat" where
  "params0 [] = (0, 0)" |
  "params0 (b # bs) =
    (let (c,s) = params0 bs in
     if b then (2*c + 3^s, Suc s) else (2*c, s))"

definition formula_of :: "bool list ⇒ nat × nat × nat" where
  "formula_of x = (length x, snd (params0 x), fst (params0 x))"

lemma params0_odd_count:
  "snd (params0 xs) = length (filter id xs)"
proof (induction xs)
  case Nil
  show ?case by simp
next
  case (Cons b bs)
  obtain c s where P: "params0 bs = (c,s)"
  by (cases "params0 bs") auto
  have S: "s = length (filter id bs)"
  using Cons.IH by (simp add: P id_def)
  show ?case by (cases b) (simp_all add: P S id_def)
qed

lemma params0_two_odd_steps:
  "params0 [True, True] = (5, 2)"
by simp
```

Injectivity

Different parity vectors produce distinct triples (k, s, c) in the representation

$$T^{(k)}(n) = \frac{3^s \cdot n + c}{2^k}.$$

```
lemma params0_injective_len:
  assumes "length xs = length ys" and "params0 xs = params0 ys"
  shows "xs = ys"
```

```

    using assms
  proof (induction xs arbitrary: ys)
    case Nil
    then show ?case by (cases ys) auto
  next
    case (Cons a xs)
    from Cons.prem1 obtain b zs where Y: "ys = b # zs"
      by (cases ys) auto
    obtain c s where X: "params0 xs = (c,s)"
      by (cases "params0 xs") auto
    obtain d t where Z: "params0 zs = (d,t)"
      by (cases "params0 zs") auto
    have E:
      "(if a then (2*c + 3^s, Suc s) else (2*c,s)) =
       (if b then (2*d + 3^t, Suc t) else (2*d,t))"
      using Cons.prem2 by (simp add: Y X Z)
    have H: "a = b"
    proof -
      have "odd (fst (if a then (2*c + 3^s, Suc s) else (2*c,s))) =
              odd (fst (if b then (2*d + 3^t, Suc t) else (2*d,t)))"
        using E by simp
      then show ?thesis by (cases a; cases b) simp_all
    qed
    have ST: "s = t"
      using E H by (cases a; cases b) auto
    have CD: "c = d"
      using E H ST by (cases a; cases b) auto
    have P: "params0 xs = params0 zs"
      using X Z CD ST by simp
    have L: "length xs = length zs"
      using Cons.prem1 by (simp add: Y)
    have "xs = zs" using Cons.IH[OF L P] .
    then show ?case using H Y by simp
  qed

lemma formula_determines_parity_on_len:
  assumes "length x = k" "length y = k"
  and "formula_of x = formula_of y"
  shows "x = y"
proof -
  have P: "params0 x = params0 y"
    using assms(3)
    by (cases "params0 x"; cases "params0 y";
        simp add: formula_of_def)
  have L: "length x = length y"
    using assms(1,2) by simp
  show ?thesis
    by (rule params0_injective_len[OF L P])
qed

```

3 Two-adic invariance

Key lemma

Adding $q \cdot 2^k$ to m preserves the first k parity bits:

$$\text{parity_vec}(m + q \cdot 2^k, k) = \text{parity_vec}(m, k).$$

Intuition

The offset $q \cdot 2^k$ is beyond the resolution of the first k steps. After one application of the Collatz map T , the offset becomes $q \cdot 2^{k-1}$ if m is even, or $3q \cdot 2^{k-1}$ if m is odd, and therefore remains a multiple of 2^{k-1} . This behaviour continues inductively through subsequent iterations.

Consequence

This invariance property enables realisability. One can tune a number to have any desired parity sequence by adding suitable multiples of 2^k , without affecting parity bits that have already been fixed.

```
lemma T_funpow_commute: "T ((T ^^ j) n) = (T ^^ j) (T n)"
proof (induction j)
  case 0
  show ?case by simp
next
  case (Suc j)
  have "T ((T ^^ Suc j) n) = T (T ((T ^^ j) n))" by simp
  also have "... = T ((T ^^ j) (T n))" by (simp add: Suc.IH)
  also have "... = (T ^^ Suc j) (T n)" by simp
  finally show ?case .
qed

lemma parity_vec_Suc:
  "parity_vec n (Suc k) = odd n # parity_vec (T n) k"
proof (rule nth_equalityI)
  show "length (parity_vec n (Suc k)) =
        length (odd n # parity_vec (T n) k)"
    by simp
next
  fix i assume iLt: "i < length (parity_vec n (Suc k))"
  then have iSk: "i < Suc k" by (simp add: parity_vec_def)
  consider (Z) "i = 0" | (S) j where "i = Suc j" "j < k"
    using iSk by (cases i) auto
  then show "parity_vec n (Suc k) ! i =
            (odd n # parity_vec (T n) k) ! i"
  proof cases
    case Z
    show ?thesis
```

```

        using Z by (simp add: parity_vec_def del: upt_Suc)
    next
        case (S j)
        show ?thesis
        using S
        by (simp add: parity_vec_def T_funpow_commute del: upt_Suc)
    qed
qed

lemma T_even_identity:
  assumes "even n"
  shows "2 * T n = n"
proof -
  obtain q where N: "n = 2*q" using assms by (elim evenE)
  show ?thesis by (simp add: N T_def)
qed

lemma T_odd_identity:
  assumes "odd n"
  shows "2 * T n = 3*n + 1"
proof -
  obtain q where N: "n = 2*q + 1" using assms by (elim oddE)
  show ?thesis by (simp add: N T_def algebra_simps)
qed

lemma collatz_affine_identity:
  "2^k * Tpow k n =
   3^(snd (params0 (parity_vec n k))) * n +
   fst (params0 (parity_vec n k))"
proof (induction k arbitrary: n)
  case 0
  show ?case by (simp add: parity_vec_def)
next
  case (Suc k)
  obtain c s where P: "params0 (parity_vec (T n) k) = (c,s)"
    by (cases "params0 (parity_vec (T n) k)") auto
  have IH: "2^k * Tpow k (T n) = 3^s * T n + c"
    using Suc.IH[of "T n"] by (simp add: P)
  have D:
    "2^(Suc k) * Tpow (Suc k) n = 3^s * (2 * T n) + 2*c"
  proof -
    have "2^(Suc k) * Tpow (Suc k) n =
      2 * (2^k * Tpow k (T n))"
      by (simp add: T_funpow_commute algebra_simps)
    also have "... = 2 * (3^s * T n + c)" by (simp only: IH)
    also have "... = 3^s * (2 * T n) + 2*c" by (simp add: algebra_simps)
    finally show ?thesis .
  qed
  show ?case

```

```

proof (cases "even n")
  case True
    have Q: "params0 (parity_vec n (Suc k)) = (2*c,s)"
      by (simp add: parity_vec_Suc P True)
    show ?thesis using D T_even_identity[OF True] by (simp add: Q)
  next
    case False
    have O: "odd n" using False by simp
    have Q: "params0 (parity_vec n (Suc k)) =
      (2*c + 3^s, Suc s)"
      by (simp add: parity_vec_Suc P False)

    have R:
      "2^(Suc k) * Tpow (Suc k) n =
      3^s * (3*n + 1) + 2*c"
      using D
      by (simp only: T_odd_identity[OF O])

    show ?thesis
      using R by (simp add: Q algebra_simps)
  qed
qed

lemma parity_vec_add_pow2_invariant:
  fixes m q k :: nat
  shows "parity_vec (m + q * 2 ^ k) k = parity_vec m k"
proof (induction k arbitrary: m q)
  case 0
  show ?case by (simp add: parity_vec_def)
next
  case (Suc k)
  let ?Δ = "q * 2 ^ Suc k"

  have head: "odd (m + ?Δ) = odd m"
    by simp

  have tail: "parity_vec (T (m + ?Δ)) k = parity_vec (T m) k"
  proof (cases "even m")
    case True
    then obtain t where m2: "m = 2*t" by (elim evenE)
    have "(m + ?Δ) div 2 = t + q * 2 ^ k"
      by (simp add: m2)
    hence "T (m + ?Δ) = t + q * 2 ^ k"
      using True by (simp add: T_def)
    moreover have "T m = t"
      using True m2 by (simp add: T_def)
    ultimately show ?thesis
      using Suc.IH[of t q] by simp
  next

```

```

case False
then obtain t where m2: "m = 2*t + 1" by (elim oddE)
have "(3*(m + ?Δ) + 1) div 2
      = (3*m + 1) div 2 + (3*q) * 2 ^ k"
proof -
  have "3*(m + ?Δ) + 1 = (3*m + 1) + (3*q) * 2 ^ Suc k"
    by simp
  moreover have "even (3*m + 1)"
    using False m2 by simp
  ultimately show ?thesis by simp
qed
hence "T (m + ?Δ) = T m + (3*q) * 2 ^ k"
  using False by (simp add: T_def)
thus ?thesis
  using Suc.IH[of "T m" "3*q"] by simp
qed

have step1: "parity_vec (m + ?Δ) (Suc k) =
             odd (m + ?Δ) # parity_vec (T (m + ?Δ)) k"
  by (simp add: parity_vec_Suc)
also have step2: "... = odd m # parity_vec (T m) k"
  using head tail by blast
also have step3: "... = parity_vec m (Suc k)"
  by (simp add: parity_vec_Suc)
finally show ?case .
qed

```

4 Every parity vector is realisable

Realisability

For any finite binary string x , there exists a natural number n whose parity sequence agrees with x :

$$\forall x. \exists n. \text{parity_vec } n \text{ (length } x) = x.$$

This means that every possible computational trace of the Collatz map actually occurs for some starting number. In particular, parity traces of arbitrary length, including incompressible traces, are realised.

Helper lemmas for modular arithmetic

The following lemmas establish basic modular-arithmetic facts that are used in the realisability construction.

```

lemma power_mod_nat:
  fixes a m n :: nat
  shows "(a ^ n) mod m = ((a mod m) ^ n) mod m"

```

```

proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have "(a ^ Suc n) mod m = (a * a ^ n) mod m" by simp
  also have "... = (((a mod m) * a ^ n) mod m)"
    by (simp add: mod_mult_left_eq)
  also have "... = (((a mod m) * ((a ^ n) mod m)) mod m)"
    by (simp add: mod_mult_right_eq)
  also have "... = (((a mod m) * (((a mod m) ^ n) mod m)) mod m)"
    by (simp add: Suc.IH)
  also have "... = (((a mod m) * ((a mod m) ^ n)) mod m)"
    by (simp add: mod_mult_right_eq)
  also have "... = ((a mod m) ^ Suc n) mod m" by simp
  finally show ?case .
qed

```

```

lemma power_mult_nat:
  fixes a :: nat
  shows "a ^ (m * n) = (a ^ m) ^ n"
proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have "a ^ (m * Suc n) = a ^ (m*n + m)" by (simp add: add.commute)
  also have "... = a ^ (m*n) * a ^ m" by (simp add: power_add)
  also have "... = (a ^ m) ^ n * a ^ m" by (simp add: Suc.IH)
  also have "... = (a ^ m) ^ Suc n" by simp
  finally show ?case .
qed

```

```

lemma pow4_mod3: "((4::nat) ^ m) mod 3 = 1"
  by (simp add: power_mod_nat)

```

```

lemma pow2_mod3_even:
  assumes "even 1"
  shows "(2 :: nat) ^ 1 mod 3 = 1"
proof -
  obtain m where L: "1 = 2*m" using assms by (erule evenE)
  have "2 ^ 1 mod 3 = (2 ^ (2*m)) mod 3" by (simp add: L)
  also have "... = (((2::nat) ^ 2) ^ m) mod 3" by (simp add: power_mult_nat)
  also have "... = (4 ^ m) mod 3" by simp
  also have "... = 1" by (rule pow4_mod3)
  finally show ?thesis .
qed

```

```

lemma pow2_mod3_odd:
  assumes "odd 1"
  shows "(2 :: nat) ^ 1 mod 3 = 2"
proof -
  obtain m where L: "1 = Suc (2*m)" using assms
  by (metis Suc_eq_plus1 oddE)
  have "2 ^ 1 mod 3 = (2 * 2 ^ (2*m)) mod 3" by (simp add: L)
  also have "... = (2 * ((2::nat) ^ 2) ^ m) mod 3"
  by (simp add: power_mult_nat)
  also have "... = (2 * ((4 ^ m) mod 3)) mod 3"
  by (simp add: mod_mult_right_eq)
  also have "... = (2 * 1) mod 3" by (simp add: pow4_mod3)
  also have "... = 2" by simp
  finally show ?thesis .
qed

lemma pow2_mod3:
  fixes 1 :: nat
  shows "(2 :: nat) ^ 1 mod 3 = (if even 1 then 1 else 2)"
  by (cases "even 1") (simp add: pow2_mod3_even, simp add: pow2_mod3_odd)

lemma choose_t_even_mod3:
  fixes m0 1 :: nat
  assumes "even 1"
  shows "∃ t ≤ 2. (m0 + t * (2 :: nat) ^ 1) mod 3 = 2"
proof -
  define t where "t = (2 - (m0 mod 3)) mod 3"
  have t_le2: "t ≤ 2" by (simp add: t_def)
  have "(m0 + t * 2 ^ 1) mod 3
    = (m0 mod 3 + t * (2 ^ 1 mod 3)) mod 3"
  by (metis mod_add_cong mod_mod_trivial mod_mult_right_eq)
  also have "... = (m0 mod 3 + t) mod 3"
  using assms by (simp add: pow2_mod3)
  also have "... = (m0 mod 3 + ((2 - (m0 mod 3)) mod 3)) mod 3"
  by (simp add: t_def)
  also have "... = 2"
  by (cases "m0 mod 3") simp_all
  finally have targ: "(m0 + t * 2 ^ 1) mod 3 = 2" .
  from t_le2 targ show ?thesis by blast
qed

lemma choose_t_odd_mod3:
  fixes m0 1 :: nat
  assumes "odd 1"
  shows "∃ t ≤ 2. (m0 + t * (2 :: nat) ^ 1) mod 3 = 2"
proof -
  define t where "t = (2 * (2 - (m0 mod 3))) mod 3"
  have t_le2: "t ≤ 2" by (simp add: t_def)
  have "(m0 + t * 2 ^ 1) mod 3

```

```

      = (m0 mod 3 + t * (2 ^ 1 mod 3)) mod 3"
    by (metis mod_add_cong mod_mod_trivial mod_mult_right_eq)
  also have "... = (m0 mod 3 + ((2 * (2 - (m0 mod 3))) mod 3) * 2)
    mod 3"
    by (simp add: t_def pow2_mod3 assms)
  also have "... = (m0 mod 3 + (4 * (2 - (m0 mod 3))) mod 3) mod 3"
    using mod_mult_right_eq by (metis (no_types, lifting)
      distrib_right mod_add_eq mod_add_left_eq mult_2_right numeral_Bit0_eq_double)
  also have "... = (m0 mod 3 + ((4 mod 3) *
    ((2 - (m0 mod 3)) mod 3)) mod 3) mod 3"
    using mod_mult_left_eq by (metis mod_mult_right_eq)
  also have "... = (m0 mod 3 + ((2 - (m0 mod 3)) mod 3)) mod 3" by simp
  also have "... = 2" by (cases "m0 mod 3") simp_all
  finally have targ: "(m0 + t * 2 ^ 1) mod 3 = 2" .
  show ?thesis using t_le2 targ by blast
qed

lemma parity_vector_realizable:
  fixes x :: "bool list"
  shows "∃n. parity_vec n (length x) = x"
proof (induction x)
  case Nil
  show ?case by (metis length_0_conv length_parity_vec)
next
  case (Cons b bs)
  obtain m0 where IH: "parity_vec m0 (length bs) = bs"
    using Cons.IH by blast
  show ?case
  proof (cases b)
    case False
    let ?n = "2*m0"
    have "parity_vec ?n (length (b # bs))
      = odd ?n # parity_vec (T ?n) (length bs)"
      by (simp add: parity_vec_Suc)
    also have "... = False # bs" by (simp add: T_def IH)
    also have "... = b # bs" by (simp add: False)
    finally show ?thesis by (intro exI[of _ ?n])
  next
    case True
    let ?l = "length bs"
    obtain t where t_le2: "t ≤ 2" and
      targ: "(m0 + t * 2 ^ ?l) mod 3 = 2"
      by (cases "even ?l")
      (use choose_t_even_mod3[of ?l m0]
        choose_t_odd_mod3[of ?l m0] in auto)
    let ?m = "m0 + t * 2 ^ ?l"
    have tail_preserved: "parity_vec ?m ?l = bs"
      using IH parity_vec_add_pow2_invariant[of m0 t ?l] by simp
    define q where "q = ?m div 3"

```

```

have m_eq: "?m = 3*q + 2"
proof -
  have "?m = 3 * (?m div 3) + (?m mod 3)" by (simp add: div_mult_mod_eq)
  also have "... = 3*q + 2" by (simp add: q_def targ)
  finally show ?thesis .
qed
let ?n = "(2 * ?m - 1) div 3"
have head: "odd ?n"
proof -
  have "?n = (2 * (3*q + 2) - 1) div 3" by (simp add: m_eq)
  also have "... = (6*q + 3) div 3" by simp
  also have "... = 2*q + 1" by simp
  finally show ?thesis by simp
qed
have tail_step: "parity_vec (T ?n) ?l = bs"
  using tail_preserved head by (simp add: m_eq T_def)
have "parity_vec ?n (length (b # bs)) =
  odd ?n # parity_vec (T ?n) ?l"
  by (simp add: parity_vec_Suc)
also have "... = True # bs" using head tail_step by simp
also have "... = b # bs" by (simp add: True)
finally show ?thesis by (intro exI[of _ ?n])
qed
qed

```

Realisability with matching successive parity

Let x be a bit vector of length $L + 1$. There is a positive natural number n such that

$$x = (n, T(n), \dots, T^{(L)}(n)) \pmod{2}$$

and

$$T^{(L+1)}(n) = T^{(L)}(n) \pmod{2}.$$

To obtain matching parities at steps L and $L + 1$, append a copy of the final bit of x to the vector. Parity-vector realisability, together with two-adic invariance, then supplies a positive starting value having this extended parity vector.

Since n is positive, every value in its trajectory is positive. If $T^{(L)}(n) = 1$, then $T^{(L+1)}(n) = 2$; if $T^{(L)}(n) = 2$, then $T^{(L+1)}(n) = 1$. In either case, the two values have opposite parities. Their equal parities therefore imply that $T^{(L)}(n) > 2$. If the trajectory had reached 1 at some step $k \leq L$, every subsequent value through step L would belong to the cycle $1, 2, 1, 2, \dots$, which would give $T^{(L)}(n) \leq 2$. This contradicts $T^{(L)}(n) > 2$. Therefore, $T^{(k)}(n) = 1$ implies $k > L$.

lemma `parity_vector_realizable_with_matching_next_parity:`
`assumes x_len: "length x = Suc L"`

```

shows "∃n>0.
      parity_vec n (Suc L) = x ∧
      odd (Tpow (Suc L) n) = odd (Tpow L n)"
proof -
  let ?y = "x @ [x ! L]"
  obtain m where m_realizes:
    "parity_vec m (length ?y) = ?y"
    using parity_vector_realizable[of ?y] by blast
  define n where "n = m + 2 ^ Suc (Suc L)"
  have y_len: "length ?y = Suc (Suc L)"
    using x_len by simp
  have m_realizes':
    "parity_vec m (Suc (Suc L)) = ?y"
    using m_realizes y_len by simp
  have invariant:
    "parity_vec n (Suc (Suc L)) =
     parity_vec m (Suc (Suc L))"
    using parity_vec_add_pow2_invariant[of m 1 "Suc (Suc L)"]
    by (simp add: n_def)
  have n_realizes:
    "parity_vec n (Suc (Suc L)) = ?y"
    using invariant m_realizes' by simp
  have n_pos: "n > 0"
    by (simp add: n_def)
  have prefix:
    "parity_vec n (Suc L) = x"
  proof -
    have "parity_vec n (Suc L) =
          take (Suc L) (parity_vec n (Suc (Suc L)))"
      by (simp add: parity_vec_def take_map)
    also have "... = take (Suc L) ?y"
      by (simp add: n_realizes)
    also have "... = x"
      using x_len by simp
    finally show ?thesis .
  qed
  have at_L:
    "odd (Tpow L n) = x ! L"
  proof -
    have x_nonempty: "x ≠ []"
      using x_len by auto

    have "odd (Tpow L n) =
          last (parity_vec n (Suc L))"
      by (simp add: parity_vec_def)
    also have "... = last x"
      by (simp add: prefix)
    also have "... = x ! (length x - 1)"
      by (rule last_conv_nth[OF x_nonempty])
  qed

```

```

    also have "... = x ! L"
    using x_len by simp
    finally show ?thesis .
qed

have at_Suc_L:
  "odd (Tpow (Suc L) n) = x ! L"
proof -
  have "odd (Tpow (Suc L) n) =
    last (parity_vec n (Suc (Suc L)))"
    by (simp add: parity_vec_def)
  also have "... = last ?y"
    by (simp add: n_realizes)
  also have "... = x ! L"
    by simp
  finally show ?thesis .
qed
show ?thesis
  using n_pos prefix at_L at_Suc_L by blast
qed

lemma T_positive:
  assumes "n > 0"
  shows "T n > 0"
proof (cases "even n")
  case True
  then obtain q where n_eq: "n = 2 * q"
    by (elim evenE)
  have "q > 0"
    using assms n_eq by simp
  then show ?thesis
    using True by (simp add: T_def n_eq)
next
  case False
  then obtain q where n_eq: "n = 2 * q + 1"
    by (elim oddE)
  then show ?thesis
    by (simp add: T_def)
qed

lemma Tpow_positive:
  assumes "n > 0"
  shows "Tpow k n > 0"
  using assms
proof (induction k arbitrary: n)
  case 0
  then show ?case by simp
next
  case (Suc k)

```

```

have iter_pos: "Tpow k n > 0"
  using Suc.IH Suc.prem by blast
have "T (Tpow k n) > 0"
  using T_positive[OF iter_pos] .
then show ?case
  by (simp add: funpow_Suc_right)
qed

lemma equal_successive_parity_imp_gt_two:
  assumes n_pos: "n > 0"
  and same_parity:
    "odd (Tpow (Suc L) n) = odd (Tpow L n)"
  shows "Tpow L n > 2"
proof -
  have step:
    "Tpow (Suc L) n = T (Tpow L n)"
  by (simp add: funpow_Suc_right)
  have iter_pos: "Tpow L n > 0"
  using Tpow_positive n_pos by blast
  have not_one: "Tpow L n  $\neq$  1"
  using same_parity step by (auto simp: T_def)
  have not_two: "Tpow L n  $\neq$  2"
  using same_parity step by (auto simp: T_def)
  show ?thesis
  using iter_pos not_one not_two by linarith
qed

lemma Tpow_one_le_two:
  "Tpow j 1  $\leq$  2"
proof -
  have "Tpow j 1 = 1  $\vee$  Tpow j 1 = 2"
  proof (induction j)
    case 0
    then show ?case by simp
  next
    case (Suc j)
    then show ?case
      by (auto simp: funpow_Suc_right T_def)
  qed
  then show ?thesis by auto
qed

lemma reaches_one_only_after_L:
  assumes n_pos: "n > 0"
  and same_parity:
    "odd (Tpow (Suc L) n) = odd (Tpow L n)"
  and reaches: "Tpow k n = 1"
  shows "k > L"
proof (rule ccontr)

```

```

assume "¬ k > L"
then have k_le: "k ≤ L" by simp
have L_decomp: "L = (L - k) + k"
  using k_le by simp
have "Tpow L n = Tpow ((L - k) + k) n"
  by (rule arg_cong[OF L_decomp])
also have "... = Tpow (L - k) (Tpow k n)"
  by (simp add: funpow_add)
also have "... = Tpow (L - k) 1"
  by (simp add: reaches)
also have "... ≤ 2"
  by (rule Tpow_one_le_two)
finally have "Tpow L n ≤ 2" .
moreover have "Tpow L n > 2"
  using equal_successive_parity_imp_gt_two[OF n_pos same_parity] .
ultimately show False by simp
qed

```

5 First arrival and parity prefixes

The following two lemmas connect an initial parity pattern with the complete trace leading to the first arrival at 1.

First, if a positive trajectory reaches 1, there is an earliest step r at which it does so. Every value before that step is greater than 1. This establishes the first-arrival condition used in the trace-specification assumption.

Second, recording more steps does not change the parity bits already recorded. For $m \leq r$, the first m bits of the length- r parity trace are exactly the length- m parity trace.

If two trajectories have different initial parity patterns of the same length, their complete traces to the first arrival at 1 are also different, provided both traces include those initial patterns.

```

lemma first_hit_exists:
  assumes n_pos: "n > 0"
  and reaches: "∃ r. Tpow r n = 1"
  shows "∃ r. Tpow r n = 1 ∧ (∀ j < r. Tpow j n > 1)"
proof -
  let ?r = "LEAST r. Tpow r n = 1"
  have hit: "Tpow ?r n = 1"
    by (rule LeastI_ex[OF reaches])
  have before: "∀ j < ?r. Tpow j n > 1"
  proof (intro allI impI)
    fix j
    assume j_lt: "j < ?r"
    have not_one: "Tpow j n ≠ 1"
    proof
      assume "Tpow j n = 1"
      then have "?r ≤ j" by (rule Least_le)
    qed
  qed

```

```

      with j_lt show False by simp
    qed
    have "Tpow j n > 0"
      by (rule Tpow_positive[OF n_pos])
    with not_one show "Tpow j n > 1" by linarith
  qed
  show ?thesis using hit before by blast
qed

lemma parity_vec_take_prefix:
  assumes "m ≤ r"
  shows "take m (parity_vec n r) = parity_vec n m"
  using assms by (simp add: parity_vec_def take_map)

```

6 Proof system setup

```

type_synonym bit = bool
type_synonym bitstring = "bit list"

```

Substring containment

We introduce a substring-containment relation as a *concrete and explicit* model of proofs that store computational trace information as literal data. Formally, a bitstring p contains a bitstring s if s occurs verbatim as a contiguous substring of p , i.e. if there exist bitstrings u and v such that $p = u@s@v$.

Remark. The choice of substring containment is deliberately strong. It provides a simple, syntactic notion of explicit information storage that is easy to reason about formally and avoids ambiguity about how information is represented inside a proof certificate. In the main barrier theorem, literal containment implies that the encoded first-arrival trace cannot be longer than the proof certificate. Injectivity makes the encodings of distinct completed traces distinct. The main theorem counts these encodings directly; it does not assume that an encoding preserves the prefix relation.

```

definition contains :: "bitstring ⇒ bitstring ⇒ bool"
  where "contains p s ⟷ (∃ u v. p = u @ s @ v)"

```

```

lemma contains_len_bound: "contains p s ==> length s <= length p"
  by (auto simp: contains_def)

```

7 The unprovability theorem

```

lemma finite_bitstrings_of_len:
  "finite {s::bitstring. length s = m}"
proof -

```

```

have fin_aux:
  "finite {s::bool list. set s <= (UNIV::bool set) ∧ length s = m}"
  by (rule finite_lists_length_eq) simp
have "{s::bitstring. length s = m}
      = {s. set s <= (UNIV::bool set) ∧ length s = m}"
  by auto
then show ?thesis using fin_aux by (simp only:)
qed

lemma many_strings_of_length:
  "card {s::bitstring. length s = m} = 2 ^ m"
proof (induction m)
  case 0
  have "{s::bitstring. length s = 0} = {[]}" by auto
  thus ?case by simp
next
  case (Suc m)
  have "{s. length s = Suc m} =
        (%b. True # b) ` {s. length s = m}
        Un (%b. False # b) ` {s. length s = m}"
    by (auto simp: length_Suc_conv)
  moreover have "(\b. True # b) ` {s. length s = m}
        Int (\b. False # b) ` {s. length s = m} = {}"
    by auto
  moreover have "inj_on (\b. True # b) {s. length s = m}"
    by (auto simp: inj_on_def)
  moreover have "inj_on (\b. False # b) {s. length s = m}"
    by (auto simp: inj_on_def)
  ultimately show ?case
    using Suc.IH card_Un_disjoint card_image
    by (smt (verit) Suc_1 Suc_pred card.infinite diff_add_zero
        mult_2 nat.discI plus_1_eq_Suc power_Suc0_right power_add
        power_eq_0_iff zero_less_one)
qed

definition compressible :: "bitstring ⇒ (bitstring ⇒ bitstring) ⇒ bool"
where
  "compressible s enc ≡ length (enc s) < length s"

definition incompressible_by :: "bitstring ⇒ (bitstring ⇒ bitstring)
⇒ bool" where
  "incompressible_by s enc ≡ ¬compressible s enc"

lemma sum_pow2_lt: "sum (%i. (2::nat) ^ i) {..

```

```

    case 0 show ?case by (simp add: finite_bitstrings_of_len)
next
  case (Suc m)
  have "{s::bitstring. length s <= Suc m}
    = {s. length s <= m} Un {s. length s = Suc m}" by auto
  thus ?case using Suc.IH finite_bitstrings_of_len by (simp add: finite_UnI)
qed

lemma card_bitstrings_le_len:
  "card {s::bitstring. length s <= m} = sum (%i. 2 ^ i) {..m}"
proof -
  let ?S = "\i. {s::bitstring. length s = i}"
  have union_eq: "{s::bitstring. length s <= m} =
    Union ((%i. ?S i) ` {..m})"
    by auto
  have fin_index: "finite ({..m}::nat set)" by simp
  have fin_each: "!!i. i <= m ==> finite (?S i)"
    by (simp add: finite_bitstrings_of_len)
  have disj: "!!i j. i <= m ==> j <= m ==> i ~= j
    ==> ?S i Int ?S j = {}"
    by auto
  have "card (Union ((%i. ?S i) ` {..m})) = sum (%i. card (?S i)) {..m}"
    by (rule card_UN_disjoint) (use fin_index fin_each disj in auto)
  also have "... = sum (%i. 2 ^ i) {..m}"
    by (simp add: many_strings_of_length)
  finally show ?thesis
    by (simp add: union_eq)
qed

```

The pigeonhole argument

Theorem. For any injective encoding, there exists an incompressible bitstring of every prescribed length.

The following theorem records the general incompressibility principle. The final barrier proof uses the underlying cardinality bounds directly on completed first-arrival traces.

```

theorem incompressible_strings_exist_for_enc:
  fixes enc :: "bitstring => bitstring"
  assumes "inj enc"
  shows "\s. length s = m & incompressible_by s enc"
proof (cases m)
  case 0
  have "length ([::bitstring]) = 0 & 0 <= length (enc [])"
    by simp
  then show ?thesis using 0 incompressible_by_def compressible_def by
  auto
next
  case (Suc r)

```

```

let ?S = "{t::bitstring. length t = Suc r}"
let ?T = "{u::bitstring. length u ≤ r}"
have finS: "finite ?S" by (simp add: finite_bitstrings_of_len)
have finT: "finite ?T" by (simp add: finite_bitstrings_le_len)
have Sm: "card ?S = 2 ^ Suc r"
  by (simp add: many_strings_of_length)
have Tm: "card ?T = sum (%i. (2::nat) ^ i) {..r}"
  by (simp add: card_bitstrings_le_len)
also have "... = sum (%i. (2::nat) ^ i) {..

```

Proof system locale

This locale states the information assumptions used in the argument. A bitstring p represents a proposed proof.

1. **Trace specification.** If p proves that a positive integer n reaches 1, and r is the first step at which it does so, then p contains an encoding of the parity trace leading to that first arrival. Explicitly, if

$$T^{(r)}(n) = 1 \quad \text{and} \quad T^{(j)}(n) > 1 \text{ for every } j < r,$$

then p contains an encoding of

$$(n, T(n), \dots, T^{(r-1)}(n)) \pmod{2}.$$

For $r = 0$, this trace is empty. Only the completed first-arrival trace is required; its earlier prefixes need not be encoded separately.

2. **Injectivity of the parity encoding.** Different parity traces have different encodings.
3. **Soundness for individual instances.** If p proves that a positive integer n reaches 1, then there exists an r such that $T^{(r)}(n) = 1$.
4. **Universal instantiation.** If p proves the Collatz conjecture, then for every positive n the same certificate p proves the instance

$$\exists r. T^{(r)}(n) = 1.$$

Motivation for the trace-specification assumption

The trace-specification assumption is motivated by two structural properties of the Collatz map.

1. **Arbitrarily large excursions.** The affine representation

$$T^{(k)}(n) = \frac{3^s n + c}{2^k}$$

allows Collatz iterates to grow arbitrarily large relative to their starting values. For example, for every $k \geq 1$, starting at $n = 2^k - 1$ gives k consecutive increasing steps and reaches $3^k - 1$. The ratio

$$\frac{3^k - 1}{2^k - 1}$$

grows without bound as k increases.

This contrasts with the map

$$U(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ (n+1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

For every $n > 1$, we have $1 \leq U(n) < n$, regardless of whether n is even or odd. Repeated application therefore reaches 1, without requiring an explicit parity trace.

2. **Injectivity of the affine representation.** In the Collatz affine representation, s counts the odd steps, while c depends on their positions. As shown earlier, for a fixed trace length k , the correspondence between parity vectors and their associated affine parameters (s, c) is injective.

Property 1 rules out the simple step-by-step descent argument used for U . The affine representation gives the exact formula for $T^{(k)}(n)$ associated with its parity vector. By property 2, its exact parameters and the parity vector determine one another at a fixed trace length. We therefore investigate

proofs that explicitly store this parity information. The trace-specification assumption requires the proof certificate to contain the encoded parity trace leading to the first arrival at 1 explicitly. The Isabelle development establishes the resulting limitation on such certificates.

```

locale Collatz_Trace_Barrier =
  fixes enc_parity :: "bool list  $\Rightarrow$  bitstring"
    and proves_reaches_one :: "bitstring  $\Rightarrow$  nat  $\Rightarrow$  bool"
    and is_collatz_proof :: "bitstring  $\Rightarrow$  bool"
  assumes enc_parity_injective: "inj enc_parity"
  assumes instance_soundness:
    "[proves_reaches_one p n; n > 0]
      $\Rightarrow \exists r. \text{Tpow } r \ n = 1$ "
  assumes first_hit_trace_specification:
    "[proves_reaches_one p n;
     n > 0;
     Tpow r n = 1;
      $\forall j < r. \text{Tpow } j \ n > 1]$ 
      $\Rightarrow \text{contains } p \ (\text{enc\_parity } (\text{parity\_vec } n \ r))$ "
  assumes collatz_proof_instances:
    "is_collatz_proof p  $\Rightarrow$ 
      $\forall n > 0. \text{proves\_reaches\_one } p \ n$ "

```

begin

If a universal certificate exists, then every chosen parity pattern of length $L + 1$ can be realised by a positive trajectory that has not reached 1 by step L . Universal instantiation and soundness guarantee that it reaches 1 later. Trace specification then requires the certificate to contain an encoding of the complete parity trace leading to that first arrival.

```

lemma stored_first_hit_extension:
  assumes p_proof: "is_collatz_proof p"
    and x_len: "length x = Suc L"
  shows " $\exists y. \text{take } (\text{Suc } L) \ y = x \wedge \text{contains } p \ (\text{enc\_parity } y)$ "
proof -
  obtain n where
    n_pos: "n > 0" and
    prefix: "parity_vec n (Suc L) = x" and
    matching: "odd (Tpow (Suc L) n) = odd (Tpow L n)"
  using parity_vector_realizable_with_matching_next_parity[OF x_len]
  by blast
  have instance_proof: "proves_reaches_one p n"
    using collatz_proof_instances[OF p_proof] n_pos by blast
  have reaches: " $\exists r. \text{Tpow } r \ n = 1$ "
    by (rule instance_soundness[OF instance_proof n_pos])
  obtain r where
    hit: "Tpow r n = 1" and
    before: " $\forall j < r. \text{Tpow } j \ n > 1$ "
  using first_hit_exists[OF n_pos reaches] by blast
  have r_gt: "r > L"

```

```

    by (rule reaches_one_only_after_L[OF n_pos matching hit])
  have prefix_length: "Suc L ≤ r" using r_gt by simp
  have extends: "take (Suc L) (parity_vec n r) = x"
    using parity_vec_take_prefix[OF prefix_length, of n] prefix by simp
  have stored: "contains p (enc_parity (parity_vec n r))"
    by (rule first_hit_trace_specification
        [OF instance_proof n_pos hit before])
  show ?thesis using extends stored by blast
qed

```

Interpretation of the main theorem

Let L be the length of a proposed universal certificate. There are 2^{L+1} prefixes of length $L + 1$. For each prefix, choose a complete parity trace that begins with that prefix and leads to the first arrival at 1. The preceding lemma guarantees that its encoding occurs in the certificate. Distinct prefixes give distinct completed traces, and injectivity gives distinct encodings. Every encoding must have length at most L because it occurs inside the certificate. But there are only $2^{L+1} - 1$ bitstrings of length at most L , giving a contradiction.

```

theorem no_finite_collatz_proof:
  assumes p_proof: "is_collatz_proof p"
  shows False
proof -
  let ?L = "length p"
  let ?S = "{x::bitstring. length x = Suc ?L}"
  let ?B = "{z::bitstring. length z ≤ ?L}"

  define f :: "bitstring ⇒ bitstring" where
    "f x = (SOME y. take (Suc ?L) y = x ∧
                        contains p (enc_parity y))" for x

  have f_properties:
    "take (Suc ?L) (f x) = x ∧ contains p (enc_parity (f x))"
  if x_in: "x ∈ ?S" for x
proof -
  have x_len: "length x = Suc ?L" using x_in by simp
  have exists_extension:
    "∃ y. take (Suc ?L) y = x ∧ contains p (enc_parity y)"
  by (rule stored_first_hit_extension[OF p_proof x_len])
  show ?thesis
    unfolding f_def by (rule someI_ex[OF exists_extension])
qed

have enc_f_inj: "inj_on (λx. enc_parity (f x)) ?S"
proof (rule inj_onI)
  fix x y

```

```

    assume x_in: "x ∈ ?S" and y_in: "y ∈ ?S"
    and equal_enc: "enc_parity (f x) = enc_parity (f y)"
    have equal_traces: "f x = f y"
    using enc_parity_injective equal_enc by (rule injD)
    have x_prefix: "take (Suc ?L) (f x) = x"
    using f_properties[OF x_in] by blast
    have y_prefix: "take (Suc ?L) (f y) = y"
    using f_properties[OF y_in] by blast
    show "x = y" using equal_traces x_prefix y_prefix by metis
qed

have encodings_fit:
  "(λx. enc_parity (f x)) ` ?S ⊆ ?B"
proof
  fix z
  assume "z ∈ (λx. enc_parity (f x)) ` ?S"
  then obtain x where x_in: "x ∈ ?S"
  and z_eq: "z = enc_parity (f x)" by blast
  have stored: "contains p (enc_parity (f x))"
  using f_properties[OF x_in] by blast
  have "length (enc_parity (f x)) ≤ length p"
  by (rule contains_len_bound[OF stored])
  then show "z ∈ ?B" by (simp add: z_eq)
qed

have finite_B: "finite ?B"
  by (rule finite_bitstrings_le_len)
have image_bound:
  "card ((λx. enc_parity (f x)) ` ?S) ≤ card ?B"
  by (rule card_mono[OF finite_B encodings_fit])
have image_card:
  "card ((λx. enc_parity (f x)) ` ?S) = card ?S"
  by (rule card_image[OF enc_f_inj])
have source_card: "card ?S = 2 ^ Suc ?L"
  by (rule many_strings_of_length)
have target_card: "card ?B = 2 ^ Suc ?L - 1"
proof -
  have "card ?B = sum (λi. (2::nat)^i) {..

```

```

    show False using impossible positive by linarith
qed

corollary no_collatz_proof_in_this_system:
  " $\neg (\exists p. \text{is\_collatz\_proof } p)$ "
  using no_finite_collatz_proof by blast

end
end

```